



Solution Design Document

PayPal Checkout + BCDC

PayPal JavaScript SDK v6 — Server-Side Architecture

PayPal · PayLater · PayPal Credit · BCDC (Guest Card) · STC

Document type	Solution Design Document (SDD)
Solution	Payment processing with PayPal Checkout, PayLater, PayPal Credit and BCDC using the PayPal Web SDK v6 with server-side architecture (BFF)
Technical components	PayPal Web SDK v6 (web-sdk/v6/core) · paypal-payments · paypal-guest-payments · One-Time Payment Sessions · STC · BFF
Functional capabilities	PayPal account payment · PayLater · PayPal Credit · BCDC (Guest Card) · server-side STC · Per-operation idempotency (createRequestId / captureRequestId)
REST APIs involved	/v1/oauth2/token · /v2/checkout/orders · /v2/checkout/orders/{id} · /v2/checkout/orders/{id}/capture · /v1/risk/transaction-contexts/{merchant_id}/{cmid}
Conventions	REST → snake_case · SDK JavaScript v6 → camelCase
Audience	Solution Architecture · Integration Engineering · Product · e-commerce · Risk · Merchant Compliance

Table of Contents

Table of Contents	2
1. Executive Summary	5
2. Business Context and Drivers.....	6
2.1 Business drivers.....	6
2.2 Regulatory and compliance drivers.....	6
3. Solution Scope.....	7
3.1 In-scope capabilities.....	7
3.2 Out-of-scope capabilities.....	7
4. Stakeholders and Audience.....	9
5. Glossary of Terms and Acronyms	10
6. Solution Overview	12
6.1 Solution components	12
6.2 Correlation identifier: the CMID	13
6.3 Payment methods under a single SDK.....	13
7. Functional Requirements.....	15
8. Non-Functional Requirements.....	16
9. Solution Architecture.....	18
9.1 Logical diagram.....	18
9.2 Non-negotiable architecture rules.....	18
9.3 Security boundary per asset.....	18
10. Prerequisites and Environment Configuration	20
10.1 Required commercial enablements.....	20
10.2 Backend credentials and configuration.....	20
10.3 Environment requirements	20
10.4 Public configuration endpoint.....	21
11. OAuth2 Authentication (client_credentials).....	22
11.1 Request.....	22
11.2 Response (relevant fields).....	22
11.3 access_token cache	22
11.4 Reference implementation (Node.js)	23
12. Client Metadata ID (CMID) Generation.....	24
12.1 Lifecycle rules.....	24
12.2 Reference implementation	24

12.3 Validation	24
13. Set Transaction Context (STC).....	26
13.1 Endpoint.....	26
13.2 Body — generic set for Retail	26
13.3 Field reference	26
13.4 Response handling — non-blocking behavior	27
13.5 Reference implementation — server-side STC inside Create Order.....	27
14. Loading the PayPal Web SDK v6	29
14.1 SDK URLs	29
14.2 Insertion into the HTML.....	29
14.3 Load detection	29
15. SDK Initialization and Eligible Methods Discovery	30
15.1 paypal.createInstance(..).....	30
15.2 sdkInstance.findEligibleMethods(..)	30
15.3 PayLater-specific details.....	31
16. HTML Structure and Session Activation Pattern	32
16.1 Merchant buttons.....	32
16.2 Session activation pattern — transient activation preservation	32
16.2.1 CORRECT pattern	32
16.2.2 INCORRECT pattern (do not implement)	32
16.3 BCDC pattern (click handler on custom element).....	32
16.4 Shape of the Create Order promise	33
16.5 Session callbacks	33
17–20. Payment Sessions	35
17. PayPal Checkout Session.....	35
18. PayLater Session	35
19. PayPal Credit Session	35
20. BCDC Session — Guest Card Checkout.....	36
20.1 Key differences vs. the other sessions.....	36
21. Order Creation and Capture.....	38
21.1 Required HTTP headers.....	38
21.2 Create Order — base payload	38
21.3 Breakdown validation rules.....	39
21.4 Capture Order.....	40

21.5 Frontend → backend contract.....	40
21.6 Backend — POST /api/orders.....	40
22. End-to-End Orchestration	42
22.1 Moment 1 — Checkout initialization (once per session).....	42
22.2 Moment 2 — Buyer click (no await before start)	42
22.3 Moment 3 — onApprove and capture	42
22.4 Mnemonic rule.....	43
23. Integration Points (Endpoint Map).....	44
24. Security Considerations	45
24.1 Credential custody (NFR-02)	45
24.2 PCI isolation (NFR-01)	45
24.3 Minimum recommended CSP (NFR-04).....	45
24.4 Logging and sensitive data handling (NFR-08)	45
25. Operational Considerations.....	47
25.1 Metrics and observability.....	47
25.2 Idempotency (NFR-06)	47
25.3 Fault tolerance	48
25.4 Sandbox → Live promotion.....	48
26. Testing Strategy and Sandbox Environment.....	50
26.1 Minimum test cases (TC-01 to TC-16).....	50
27. REST vs SDK Naming Conventions.....	52
28. Assumptions, Dependencies, and Constraints	53
28.1 Assumptions.....	53
28.2 External dependencies.....	53
28.3 Constraints	54
29. Risks and Mitigations	55
30. Acceptance Criteria and Pre-production Checklist	57
Appendix A — Common Errors Diagnostic	59
Appendix B — Limitations and Future Work	61
B.1 Known limitations	61
B.2 Suggested future work	61

Part I — Solution Context

1. Executive Summary

This Solution Design Document defines the integration of PayPal Checkout + Branded Card-Direct Checkout (BCDC) on top of the PayPal JavaScript SDK v6 (web-sdk/v6/core) in a web checkout experience with server-side architecture. The solution enables four payment methods under a single SDK and a single session activation pattern:

- PayPal — payment with the buyer's PayPal account.
- PayLater — PayPal-financed installment payment in eligible markets.
- PayPal Credit — PayPal credit line in eligible markets.
- BCDC — credit or debit card payment as guest (no PayPal account), with capture UI hosted by PayPal.

The architecture follows the Backend-for-Frontend (BFF) pattern: the buyer's browser interacts with the PayPal SDK and with a private merchant backend, and the latter is the only component authorized to talk to the PayPal REST API. The `client_secret` and `merchant_id` stay in the backend; the frontend receives only the public `client_id`.

The solution targets merchants that require:

- Broad coverage of payment methods with a single SDK and minimal maintenance surface.
- Isolation of sensitive credentials in the backend, removing `client_secret` from the browser.
- Optimized conversion via the transient activation-based pattern, which prevents pop-up blocking in modern browsers.
- Risk traceability and correlation via a Client Metadata ID (CMID) consistent across frontend, backend, STC, and the Create/Capture Order headers.
- Transactional idempotency via independent PayPal-Request-Id keys per operation (`createRequestId`, `captureRequestId`) for safe retries against network failures.

This document describes the end-to-end solution: business context, requirements, architecture, detailed design per component, orchestration, security, operations, testing, risks, and acceptance criteria.

2. Business Context and Drivers

2.1 Business drivers

Driver	Description
Payment-method coverage with a single SDK	SDK v6 exposes PayPal, PayLater, PayPal Credit, and BCDC under a single instance (createInstance) and a homogeneous session API. This reduces maintenance cost versus integrating each method separately.
Conversion and user experience	The transient activation-based activation pattern preserves the user gesture that triggers the session, allowing PayPal to open its pop-up or full-page modal without the browser blocking it.
Reduced friction for the guest buyer	BCDC enables credit/debit card payment without forcing the buyer to create a PayPal account. The capture UI is hosted by PayPal: the merchant only renders a button.
Credential isolation	The server-side architecture moves client_secret and merchant_id out of the browser, eliminating the 'secret leaked in HTML' incident category typical of legacy integrations.
Financing methods with no additional code	PayLater and PayPal Credit are exposed via APIs symmetrical to PayPal Checkout's. Market eligibility is discovered via findEligibleMethods, with no manual configuration required from the merchant.

2.2 Regulatory and compliance drivers

Driver	Description
PCI DSS	In all flows covered by this SDD, the merchant does not process, transmit, or store buyer card account data: PayPal/PayLater/Credit do not involve cards from the merchant's perspective, and in BCDC capture occurs entirely in the PayPal-hosted UI. This keeps scope under SAQ A.
Secret custody	The client_secret resides only in the backend, managed by a secrets manager. This facilitates compliance with internal credential-handling policies and reduces audit scope.
Local regulations for PayLater / Credit	Presentation of financing to the buyer is subject to local regulations. The PayLater/Credit flow UI is hosted by PayPal and meets the requirements of the market in which the SDK detects eligibility.

3. Solution Scope

3.1 In-scope capabilities

Capability	Description
PayPal account payment (One-Time)	Charge against the buyer's PayPal account via <code>createPayPalOneTimePaymentSession</code> .
PayLater payment	PayLater financing via <code>createPayLaterOneTimePaymentSession</code> . Market and product eligibility is discovered with <code>findEligibleMethods</code> and <code>paymentMethods.getDetails('paylater')</code> .
PayPal Credit payment	PayPal credit line via <code>createPayPalCreditOneTimePaymentSession</code> in eligible markets.
BCDC — Guest Card Checkout	Credit/debit card payment as guest via <code>createPayPalGuestOneTimePaymentSession</code> . The capture UI is hosted by PayPal; the merchant mounts the <code><paypal-basic-card-container></code> + <code><paypal-basic-card-button></code> custom elements and attaches the click listener to the button.
Activation pattern with transient activation	Building the Create Order promise without <code>await</code> inside the click handler, to preserve the user gesture and avoid pop-up blocking.
Set Transaction Context (STC)	Sending buyer context to the PayPal risk engine before each Create Order. Non-blocking operation.
Transactional idempotency	Safe retries via independent <code>PayPal-Request-Id</code> keys per operation (<code>createRequestId</code> for Create, <code>captureRequestId</code> for Capture), persisted server-side.
Post-approval capture	Payment capture in <code>onApprove</code> via <code>POST /v2/checkout/orders/{id}/capture</code> .
Enriched post-capture query	<code>GET /v2/checkout/orders/{id}</code> to obtain enriched <code>payment_source</code> , capture identifiers, and reconciliation data.

3.2 Out-of-scope capabilities

Category	Detail
Advanced card capabilities	Capture in merchant-owned iframes, explicit control of buyer step-up authentication and its outcome, installments, and card reuse across sessions are out of scope.

Category	Detail
Additional risk telemetry	SDK v6 automatically injects the required telemetry; the merchant does not integrate additional Fraudnet scripts.
Back-office operations	Webhooks, refunds, settlement, disputes, and financial reconciliation are out of scope. They require a complementary SDD.
Other payment methods	Local APMs and recurring subscriptions can be integrated with the same SDK v6 in complementary solutions.
Capture modes other than CAPTURE	AUTHORIZE with deferred capture is out of scope.

4. Stakeholders and Audience

Role	Responsibility regarding this SDD
Solution Architect	Validates the architecture, ensures consistency with the merchant's portfolio, and approves the document.
Product Manager (e-commerce)	Validates that the functional scope covers business requirements: payment-method coverage, target markets, and checkout experience.
Integration Engineer (PayPal)	Supports the merchant in the implementation, validates commercial configurations (PayLater/Credit enablement, BCDC, STC), and assists with migration from legacy integrations.
Integration Engineer / Tech Lead (merchant)	Leads the technical implementation and owns the frontend and backend code.
Merchant development team	Implements and maintains the integration.
Risk and Anti-Fraud team (merchant)	Defines the additional_data field set for STC, monitors fraud metrics, and validates correct CMID propagation.
PCI Compliance team (merchant)	Verifies that the integration maintains SAQ A scope and that there are no paths by which buyer card account data reaches the merchant's infrastructure.
QA team (merchant)	Executes the test plan in Sandbox prior to promotion to Live.
Operations (merchant)	Monitors the checkout in production, manages alerts and incidents.

5. Glossary of Terms and Acronyms

Term	Definition
PayPal Web SDK v6	Version 6 of the PayPal JavaScript SDK, distributed from https://www.paypal.com/web-sdk/v6/core (Live) or https://www.sandbox.paypal.com/web-sdk/v6/core (Sandbox). Loaded via <code><script async src="..."></code> with no query-string parameters.
createInstance	SDK v6 function (<code>window.paypal.createInstance({...})</code>) that returns an instance configured with <code>clientId</code> , <code>components</code> , <code>pageType</code> , and <code>locale</code> . It is the first mandatory step before any payment session.
findEligibleMethods	Method on the SDK instance that returns the payment methods eligible for a given <code>currencyCode</code> . Used to decide which buttons to render in the UI.
One-Time Payment Session	Non-recurring payment session. SDK v6 exposes a distinct constructor per payment method (<code>createPayPalOneTimePaymentSession</code> , <code>createPayLaterOneTimePaymentSession</code> , <code>createPayPalCreditOneTimePaymentSession</code> , <code>createPayPalGuestOneTimePaymentSession</code>). All share the same shape: <code>onApprove</code> , <code>onCancel</code> , <code>onError</code> callbacks (and <code>onWarn</code> in BCDC) and a <code>start(...)</code> method.
session.start(options, orderPromise)	Method that activates the session and opens the PayPal-hosted UI. Receives an options object (with <code>presentationMode</code>) and the Create Order promise. The promise must NOT be awaited when passed. The promise must resolve to <code>{ orderId: <ORDER_ID> }</code> .
presentationMode	options attribute that controls how PayPal presents its UI. Typical values: <code>'auto'</code> , <code>'modal'</code> , <code>'popup'</code> .
<paypal-basic-card-button> / <paypal-basic-card-container>	Custom elements registered by SDK v6 when the <code>'paypal-guest-payments'</code> component is enabled. They are the canonical elements to activate the BCDC session.
Transient activation	Browser state indicating that the user has performed a recent gesture (typically a click). Consumed when a window is opened or an operation requiring implicit consent is executed. <code>await</code> before invoking <code>session.start</code> consumes the state and browsers will block the pop-up.
Order	REST resource (<code>/v2/checkout/orders</code>) representing the intent to charge: amount, breakdown, items, buyer. Identified by <code>order_id</code> .
Capture	Operation that executes the actual charge (POST <code>/v2/checkout/orders/{id}/capture</code>).
access_token	OAuth2 Bearer token that the merchant backend uses to authenticate with the PayPal REST API. Must never be sent to the browser.

Term	Definition
client_credentials	OAuth2 grant type used by this integration to obtain the access_token. response_type=id_token is not requested; SDK v6 does not require id_token to initialize.
CMID	Client Metadata ID. Alphanumeric identifier of up to 32 characters with no hyphens, generated once per checkout session. Acts as the correlation thread between the frontend, the STC URL, and the PayPal-Client-Metadata-Id headers of Create Order and Capture Order.
STC	Set Transaction Context. PUT /v1/risk/transaction-contexts/{merchant_id}/{cmid} endpoint by which the merchant sends buyer context to PayPal Risk before each Create Order. Non-blocking.
PayPal-Client-Metadata-Id	HTTP header that the backend injects in Create Order and Capture Order. Contains the session's CMID.
PayPal-Request-Id	Per-operation idempotency HTTP header. Each operation (Create Order, Capture Order) has its own key (createRequestId, captureRequestId). The same key is reused only for retries of the same request.
BFF	Backend-for-Frontend. Architectural pattern in which the merchant backend exposes only the endpoints required for the checkout flow, custodies the secrets, and acts as an authenticated proxy to the PayPal REST API.
BCDC	Branded Card-Direct Checkout. PayPal product to process credit/debit cards as guest, with capture UI hosted by PayPal. Must not be confused with ACDC.
pageType	createInstance parameter that tells PayPal the type of page where the SDK operates ('checkout', 'product-details', 'cart', etc.). For this integration the value is 'checkout'.
locale	createInstance parameter in BCP-47 format ('es-MX', 'en-US', 'pt-BR', etc.). Defines the language of the PayPal-hosted UI.
currencyCode	findEligibleMethods parameter in ISO 4217 format. Determines payment-method eligibility per market.
NFR	Non-Functional Requirement.

Part II — Solution Definition

6. Solution Overview

The solution comprises three planes with strictly delimited responsibilities and a correlation identifier (CMID) that traverses all three throughout the checkout session.

Plane	Responsibility	What it does NOT do
Buyer browser	Generates the CMID, loads SDK v6, calls <code>createInstance</code> and <code>findEligibleMethods</code> , instantiates the payment sessions, renders the buttons, fires <code>session.start(...)</code> with the Create Order promise, and reacts to callbacks.	Never knows <code>CLIENT_SECRET</code> , <code>access_token</code> , or <code>MERCHANT_ID</code> . Never calls the PayPal REST API directly for business operations.
Merchant backend	Custodies the credentials, obtains the <code>access_token</code> via OAuth2 <code>client_credentials</code> , exposes proxy endpoints to PayPal, injects the PayPal-Request-Id headers (with separate idempotency keys for Create and Capture) and PayPal-Client-Metadata-Id, executes server-side STC before each Create Order, and builds the payload from the authenticated cart state.	Never receives or manipulates buyer card account data. Never exposes <code>access_token</code> , <code>client_secret</code> , or <code>merchant_id</code> to the frontend. Never accepts order payload, amount, items, or breakdown sent from the client.
PayPal REST API	Processes orders, captures, and risk context (STC).	Is the single transactional source of truth.

6.1 Solution components

Component	Plane	Function
PayPal Web SDK v6	Browser	Renders the PayPal-hosted UI, manages the payment-session lifecycle and communication with <code>paypal.com</code> .
Merchant buttons	Browser	Merchant-owned <code><button></code> HTML elements that fire <code>session.start(...)</code> . Brand and style belong to the merchant.
OAuth2 Service	Backend	Obtains and caches the <code>access_token</code> via <code>client_credentials</code> .

Component	Plane	Function
Orders Proxy	Backend	Creates, queries, and captures orders. Injects idempotency and correlation headers.
STC Caller	Backend	Calls /v1/risk/transaction-contexts server-side inside the POST /api/orders handler, before Create Order. Does not expose a public endpoint to the frontend.
PayPal REST API	PayPal	Actual processing.

6.2 Correlation identifier: the CMID

The CMID is an alphanumeric identifier of up to 32 characters with no hyphens, generated once per checkout session. It traverses all three planes:

```

Navegador                                Backend del comercio                                API PayPal
1. genera CMID
2. CMID → body._cmid de /api/orders
   (server-side, en orden estricto)
   PUT /v1/risk/transaction-contexts/{merchant_id}/{cmid}
(STC, no bloqueante)
3. CMID → body._cmid de /api/orders/:id/capture
   → PayPal-Client-Metadata-Id → POST /v2/checkout/orders
   → PayPal-Client-Metadata-Id → POST /v2/checkout/orders/{id}/capture

```

⚠ Without a CMID consistent across the three planes, PayPal Risk cannot correlate STC context with the real transaction, which degrades the quality of the risk evaluation.

💡 Fraudnet in JSv6: Unlike legacy integrations, SDK v6 automatically injects the required telemetry. The merchant must not inject additional Fraudnet scripts. The CMID is propagated via backend HTTP headers, not via SDK <script> attributes.

6.3 Payment methods under a single SDK

Method	Session constructor	Required component in createInstance
PayPal	sdkInstance.createPayPalOneTimePaymentSession({ onApprove, onCancel, onError })	'paypal-payments'
PayLater	sdkInstance.createPayLaterOneTimePaymentSession({ onApprove, onCancel, onError })	'paypal-payments'
PayPal Credit	sdkInstance.createPayPalCreditOneTimePaymentSession({ onApprove, onCancel, onError })	'paypal-payments'

Method	Session constructor	Required component in createInstance
BCDC (Guest)	<code>sdkInstance.createPayPalGuestOneTimePaymentSession({ onApprove, onCancel, onWarn, onError })</code>	'paypal-guest-payments'

 The components array passed to createInstance must explicitly include the components matching the methods the merchant will expose. For a complete integration: ['paypal-payments', 'paypal-guest-payments'].

7. Functional Requirements

ID	Requirement	Priority
FR-01	The system must allow the buyer to complete a charge using their PayPal account via <code>createPayPalOneTimePaymentSession</code> .	Required
FR-02	The system must allow the buyer to complete a credit/debit card charge as guest via BCDC (<code>createPayPalGuestOneTimePaymentSession</code>), with card data capture delegated entirely to the PayPal-hosted UI.	Required
FR-03	The system must offer PayLater and PayPal Credit when <code>findEligibleMethods</code> reports them as eligible for the session's currency and market.	Required (when applicable)
FR-04	The system must create each order with intent: "CAPTURE" and a mathematically consistent breakdown (<code>item_total + tax_total + shipping - discount = amount.value</code>).	Required
FR-05	The system must execute payment capture in <code>onApprove</code> , via a POST <code>/v2/checkout/orders/{id}/capture</code> call executed by the merchant backend.	Required
FR-06	The system must activate each payment session preserving the browser's transient activation: the Create Order promise must NOT be awaited before being passed to <code>session.start(...)</code> .	Required
FR-07	The system must transmit buyer context to PayPal Risk before each Create Order, without blocking the checkout in case of an STC error.	Required
FR-08	The system must generate a unique CMID per checkout session and propagate it as the <code>PayPal-Client-Metadata-Id</code> header in Create Order and Capture Order, and as a URL segment in the STC call.	Required
FR-09	The system must allow buyer retries within the same checkout session without regenerating the CMID.	Required
FR-10	The system must enrich the post-capture response via GET <code>/v2/checkout/orders/{id}</code> to obtain capture identifiers and reconciliation data.	Required
FR-11	The system must display a clear and actionable message to the buyer when a payment fails (<code>onError</code>) or is canceled by the buyer (<code>onCancel</code>).	Required
FR-12	The system must degrade gracefully when a payment method is not eligible: the corresponding button is not rendered and the UI does not present invalid options to the buyer.	Required

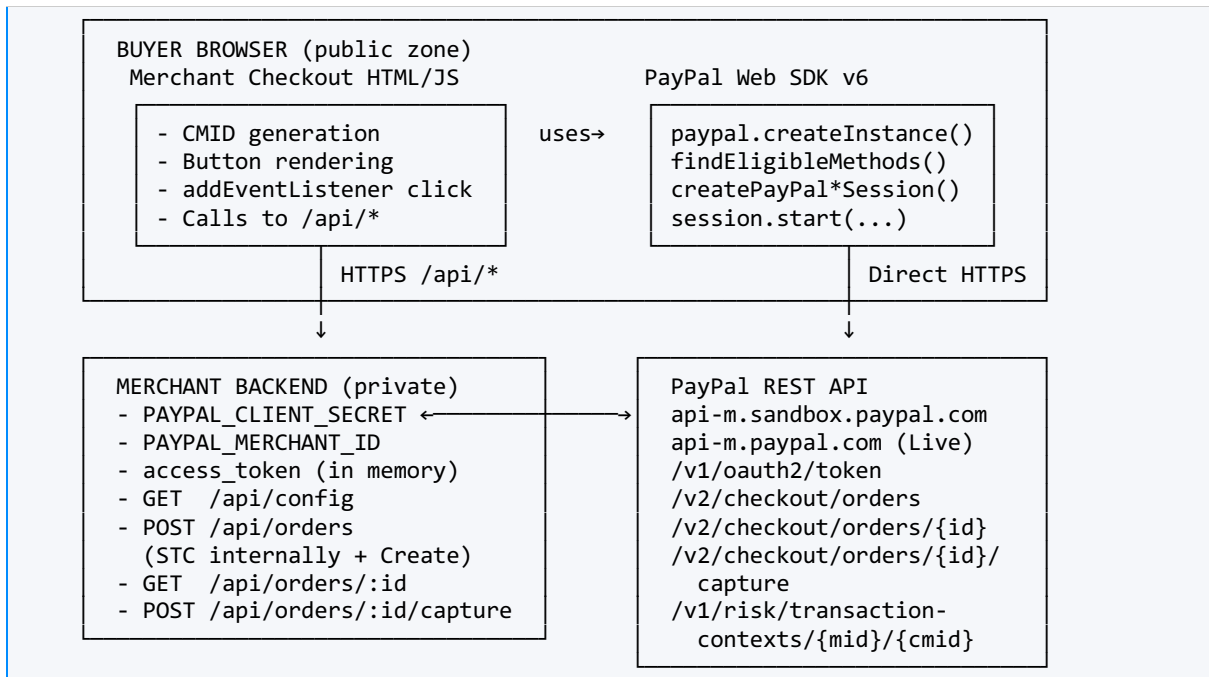
8. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security — PCI DSS	The merchant must not process, transmit, or store buyer card account data at any point of the flow. The integration must allow compliance under SAQ A.
NFR-02	Security — secrets	PAYPAL_CLIENT_SECRET, PAYPAL_MERCHANT_ID, and access_token must never be transmitted to the browser, stored in versioned source code, or appear in logs.
NFR-03	Security — transport	All communications (browser ↔ backend, backend ↔ PayPal) must use TLS 1.2 or higher.
NFR-04	Security — CSP	The site must declare a Content Security Policy that explicitly lists the PayPal origins: www.paypal.com, www.sandbox.paypal.com, www.paypalobjects.com, c.paypal.com, api-m.paypal.com.
NFR-05	Availability	Outage of ancillary services (STC) must not interrupt the payment flow. STC operates non-blocking.
NFR-06	Idempotency	The system must guarantee that retries of Create Order or Capture Order do not generate duplicate orders or captures, via PayPal-Request-Id, using independent keys per operation (createRequestId for Create, captureRequestId for Capture).
NFR-07	Latency (p95)	Total time between the button click and the response to the user must not exceed the merchant's internal SLOs. The access_token cache (§11.3) and non-blocking STC processing are essential to meet it.
NFR-08	Traceability	The system must log order_id, capture_id, status, PayPal error code, and CMID, without logging backend secrets.
NFR-09	Internationalization	The SDK must be initialized with the locale matching the target market, and error messages must be translated into the buyer's language.
NFR-10	Accessibility	Merchant buttons that activate sessions must meet WCAG 2.1 AA: visible focus, accessible labels, sufficient contrast, minimum touch area 44×44 px.
NFR-11	Scalability	The backend must cache the access_token up to 90% of expires_in to avoid saturating /v1/oauth2/token under load.
NFR-12	Browser compatibility	The integration must execute findEligibleMethods before rendering any button and omit methods that are not eligible.

ID	Category	Requirement
NFR-13	Auditability	Every transaction must be reconstructible from the logs: CMID, createRequestId, captureRequestId, order_id, invoice_id, custom_id.
NFR-14	Transient activation preservation	The button's click handler must not contain await before invoking session.start(...). The Create Order promise must be built and immediately passed to the SDK.

9. Solution Architecture

9.1 Logical diagram



9.2 Non-negotiable architecture rules

1. PAYPAL_CLIENT_SECRET resides only in backend environment variables. Never versioned, never sent to the browser.
2. The browser never calls api-m.paypal.com directly for business operations. All REST communication goes through the merchant backend.
3. The browser never receives the access_token. It receives only the public client_id (via GET /api/config) and the identifiers returned by proxy calls (order_id, capture_id).
4. Buyer card account data never touches the merchant. In BCDC the capture occurs entirely in the PayPal-hosted UI.
5. Cart, buyer, and shipping address data are never hardcoded in frontend code. They come from the authenticated session and the backend cart state.
6. The backend does not return sensitive fields (client_secret, access_token, merchant_id) to the frontend under any circumstance.

9.3 Security boundary per asset

Asset	Frontend	Backend	PayPal API
CLIENT_ID	Yes	Yes	—
CLIENT_SECRET	No	Yes (env)	—
MERCHANT_ID	No	Yes (env)	—
access_token	No	Yes (memory)	—
Card account data	PayPal-hosted UI (BCDC)	No	Yes
order_id	Yes (reference)	Yes	Yes
capture_id	Yes (reference)	Yes	Yes
CMID	Yes (generates)	Yes (receives in body)	Yes (header)

10. Prerequisites and Environment Configuration

10.1 Required commercial enablements

Capability	Aplica a
PayPal Checkout (One-Time Payments)	Payment with PayPal account.
BCDC (Branded Card-Direct Checkout)	Credit/debit card payment as guest.
PayLater	When the merchant wishes to offer PayLater in eligible markets.
PayPal Credit	When the merchant wishes to offer the PayPal credit line.
Set Transaction Context (STC)	Acceso al endpoint /v1/risk/transaction-contexts.

10.2 Backend credentials and configuration

Variable de entorno	Origen	Visibilidad
PAYPAL_CLIENT_ID	PayPal Developer Dashboard → merchant app.	Public (can be exposed to the frontend via GET /api/config).
PAYPAL_CLIENT_SECRET	PayPal Developer Dashboard → merchant app.	Secret — backend only.
PAYPAL_MERCHANT_ID	Merchant account profile in PayPal.	Private — backend only (required for STC).
PAYPAL_API_BASE	https://api-m.sandbox.paypal.com (Sandbox) · https://api-m.paypal.com (Live).	Privada.
PAYPAL_SDK_URL	https://www.sandbox.paypal.com/web-sdk/v6/core (Sandbox) · https://www.paypal.com/web-sdk/v6/core (Live).	Public (can be exposed to the frontend).

 Sandbox and Live credentials and URLs are different. PAYPAL_API_BASE, PAYPAL_SDK_URL, PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET, and PAYPAL_MERCHANT_ID must correspond to the same environment. A mismatch produces 401 Unauthorized on the first OAuth call or opaque errors in SDK initialization.

10.3 Environment requirements

- TLS / HTTPS required in production. SDK v6 does not operate over HTTP.
- Content Security Policy (CSP): script-src https://www.paypal.com https://www.paypalobjects.com https://c.paypal.com; frame-src https://www.paypal.com; connect-src https://api-m.paypal.com https://api-m.sandbox.paypal.com (plus Sandbox equivalents when applicable).
- Web Crypto API support in the browser (for crypto.randomUUID()). To support legacy browsers, implement a fallback (see §12.2).
- Pop-up support from the merchant domain. The transient activation-based activation pattern (§16) is designed so the browser allows the pop-up without requiring user configuration.

10.4 Public configuration endpoint

```
GET /api/config
→ 200 OK
{
  "clientId": "<PAYPAL_CLIENT_ID>",
  "sdkUrl": "<PAYPAL_SDK_URL>"
}
```

 This endpoint must never return client_secret, access_token, or merchant_id. Automated tests must validate that the response does not contain these keys.

Part III — Detailed Design

11. OAuth2 Authentication (client_credentials)

The merchant backend is the only component authorized to speak OAuth2 with PayPal. The purpose of this step is to obtain a Bearer access_token for the backend to invoke the PayPal REST API.

🔒 Unlike integrations based on the classic SDK, this solution does not require id_token. SDK v6 initializes only with the public client_id; there is no data-sdk-client-token attribute. The grant is strictly client_credentials.

11.1 Request

```
POST {PAYPAL_API_BASE}/v1/oauth2/token
Authorization: Basic <BASE64(CLIENT_ID:CLIENT_SECRET)>
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
```

11.2 Response (relevant fields)

```
{
  "access_token": "<ACCESS_TOKEN>",
  "expires_in": 32400,
  "token_type": "Bearer"
}
```

11.3 access_token cache

Práctica	Justificación
Cache the access_token in the backend process memory up to 90% of expires_in and refresh it proactively.	Avoids unnecessary calls to /v1/oauth2/token and reduces latency on the Create Order critical path.
No persistir el access_token en disco, base de datos ni logs.	Bearer token with transactional power; its exposure compromises the account.
Refresh immediately upon a 401 from any call and retry once.	Covers the edge case of a token revoked by credential rotation.

Práctica	Justificación
Invalidate the cache upon a manual rotation of CLIENT_SECRET.	A rotation leaves the cached token invalid; the backend must force a refresh on the next call.

11.4 Reference implementation (Node.js)

```
let cachedToken = null;
let cachedTokenExpiresAt = 0;

async function getAccessToken() {
  const now = Date.now();
  if (cachedToken && now < cachedTokenExpiresAt) return cachedToken;

  const credentials = Buffer.from(
    `${process.env.PAYPAL_CLIENT_ID}:${process.env.PAYPAL_CLIENT_SECRET}`
  ).toString("base64");

  const response = await fetch(`${process.env.PAYPAL_API_BASE}/v1/oauth2/token`, {
    method: "POST",
    headers: { "Authorization": `Basic ${credentials}`,
      "Content-Type": "application/x-www-form-urlencoded" },
    body: "grant_type=client_credentials"
  });

  if (!response.ok) throw new Error(`OAuth failed: ${response.status}`);

  const json = await response.json();
  cachedToken = json.access_token;
  cachedTokenExpiresAt = now + (json.expires_in * 1000 * 0.9);
  return cachedToken;
}
```

12. Client Metadata ID (CMID) Generation

The CMID is a unique alphanumeric identifier of up to 32 characters with no hyphens that acts as a correlation thread between: the STC endpoint URL, the PayPal-Client-Metadata-Id header sent to Create Order, and the PayPal-Client-Metadata-Id header sent to Capture Order.

12.1 Lifecycle rules

Rule	Detail
Unique per checkout session	Generated only once when initializing the checkout page.
Persistent across retries	If the buyer fails a payment and retries within the same session, the CMID is not regenerated.
Persistent across payment methods	If the buyer alternates between PayPal, PayLater, Credit, and BCDC in the same session, the CMID is preserved.
Regenerated on a new transaction	Only after the previous transaction is definitively completed or canceled is a new CMID generated in a new checkout session.
Propagated server-side by the backend	The frontend delivers it to the backend in the Create Order/Capture Order body; the backend forwards it as an HTTP header to PayPal.

12.2 Reference implementation

```
function generateCMID() {
  if (typeof crypto !== "undefined" && typeof crypto.randomUUID === "function") {
    return crypto.randomUUID().replace(/-/g, ""); // 32 caracteres hex
  }
  return "xxxxxxxxxx4xxxxyxxxxxxxxxxxxxxxx".replace(/[xy]/g, (c) => {
    const r = (Math.random() * 16) | 0;
    return (c === "x" ? r : (r & 0x3) | 0x8).toString(16);
  });
}
const cmid = generateCMID();
```

12.3 Validation

```
const isValidCMID = (cmid) => {
  typeof cmid === "string" && cmid.length >= 1 && cmid.length <= 32
  && /^[0-9a-zA-Z]+$/.test(cmid);
};
```


💡 The backend must apply this validation to the `_cmid` received from the frontend before propagating it as an HTTP header to PayPal. A malformed CMID must be rejected with 400 Bad Request.

13. Set Transaction Context (STC)

STC allows the merchant to send buyer context to PayPal Risk before each Create Order. It runs server-side inside the POST /api/orders handler, sequenced before the call to POST /v2/checkout/orders.

13.1 Endpoint

```
PUT {PAYPAL_API_BASE}/v1/risk/transaction-contexts/{MERCHANT_ID}/{CMID};
Authorization: Bearer {ACCESS_TOKEN};
Content-Type: application/json
```

13.2 Body — generic set for Retail

```
{
  "additional_data": [
    { "key": "sender_account_id", "value": "<ID_DEL_COMPRADOR_EN_LA_PLATAFORMA;" },
    { "key": "sender_first_name", "value": "<NOMBRE_DEL_COMPRADOR;" },
    { "key": "sender_last_name", "value": "<APELLIDO_DEL_COMPRADOR;" },
    { "key": "sender_email", "value": "<EMAIL_DEL_COMPRADOR;" },
    { "key": "sender_phone", "value": "<TELEFONO_SOLO_DIGITOS;" },
    { "key": "sender_country_code", "value": "<PAIS_ISO_ALPHA2;" },
    { "key": "sender_create_date", "value": "<FECHA_DE_ALTA_DEL_USUARIO;" },
    { "key": "highrisk_txn_flag", "value": "0" },
    { "key": "vertical", "value": "<VERTICAL_DEL_NEGOCIO;" }
  ]
}
```

13.3 Field reference

Field	Type	Description	Accepted values
sender_account_id	string	Unique buyer ID on the merchant's platform.	Alphanumeric, stable across sessions.
sender_first_name	string	Registered first name of the buyer.	Alphanumeric.
sender_last_name	string	Registered last name of the buyer.	Alphanumeric.
sender_email	string	Validated buyer email.	RFC 5322 format.
sender_phone	string	Buyer phone, digits only, no formatting.	[0-9]+

Field	Type	Description	Accepted values
sender_country_code	string	Buyer country in ISO 3166-1 Alpha-2.	MX, US, BR, etc.
sender_create_date	string	Buyer registration date on the merchant's platform.	yyyy-mm-ddThh:mm:ssZ yyyy-mm-dd yyyyymmdd
highrisk_txn_flag	string	Indicates whether the transaction is high-risk.	"0" = normal, "1" = high risk.
vertical	string	Business vertical.	Retail, Travel, Gaming, etc.

 Industry packs: The set above is the generic one for Retail. Verticals like Travel, OTAs, Financial Services, Gaming, and regulated platforms require additional fields. Request the matching industry pack from your PayPal Integration Engineer.

13.4 Response handling — non-blocking behavior

Status	Meaning	Action
200	OK. Context registered.	Continue with Create Order.
400	Invalid body.	Log the error with detail, continue with Create Order.
401	No permissions or token expired.	Refresh token, log, continue with Create Order.
5xx	PayPal internal error.	Log, continue with Create Order.

 STC can never block the checkout. An STC failure reduces the quality of the risk evaluation but does not prevent processing the transaction.

13.5 Reference implementation — server-side STC inside Create Order

```
// Helper function callSTC in the backend
async function callSTC({ cmid, additionalData }) {
  const accessToken = await getAccessToken();
  const response = await fetch(
    `${process.env.PAYPAL_API_BASE}/v1/risk/transaction-
    contexts/${process.env.PAYPAL_MERCHANT_ID}/${cmid}`,
    { method: "PUT",
      headers: { "Authorization": `Bearer ${accessToken}`, "Content-Type":
"application/json" },
      body: JSON.stringify({ additional_data: additionalData }) }
  );
  if (!response.ok) throw new Error(`STC failed: ${response.status}`);
}
```

```
// Usage inside POST /api/orders
try {
  await callSTC({
    cmid: _cmid,
    additionalData: buildAdditionalDataFromTrustedState(req.user)
  });
} catch (err) {
  logger.warn({ err, cmid: _cmid }, "STC failed; continuing checkout");
}
// Continue with Create Order regardless of the STC result
```

 STC runs before each Create Order, regardless of payment method. The frontend does not invoke STC directly; it only sends the _cmid to the backend in the POST /api/orders body.

14. Loading the PayPal Web SDK v6

SDK v6 is loaded with a `<script async>` directly in the checkout HTML. It does not accept query-string parameters like the classic SDK.

14.1 SDK URLs

Environment	URL
Sandbox	https://www.sandbox.paypal.com/web-sdk/v6/core
Live	https://www.paypal.com/web-sdk/v6/core

14.2 Insertion into the HTML

```
<script async src="&lt;PAYPAL_SDK_URL&gt;"&gt;&lt;/script&gt;
```

🔒 Do not add data-* attributes to the `<script>`. SDK v6 is not configured via tag attributes; all configuration goes through `paypal.createInstance(...)`.

14.3 Load detection

```
function loadSDK(sdkUrl) {  
  return new Promise((resolve, reject) => {  
    const s = document.createElement("script");  
    s.async = true;  
    s.src = sdkUrl;  
    s.onload = () => resolve(window.paypal);  
    s.onerror = () => reject(new Error("PayPal SDK failed to load"));  
    document.body.appendChild(s);  
  });  
}
```

⚠️ Switching between Sandbox and Live requires reloading the page. SDK v6 registers custom elements and these cannot be re-registered in the same session. Any logic that intends to alternate environments dynamically must trigger a full reload.

15. SDK Initialization and Eligible Methods Discovery

15.1 paypal.createInstance(...)

```
const sdkInstance = await window.paypal.createInstance({
  clientId:      "<PAYPAL_CLIENT_ID>",
  components:    ["paypal-payments", "paypal-guest-payments"],
  pageType:      "checkout",
  locale:        "<LOCALE_BCP47>",
  clientMetadataId: cmid // opcional pero recomendado
});
```

Parameter	Value	Notes
clientId	<PAYPAL_CLIENT_ID>	Received from the backend in GET /api/config. It is not a secret: it may appear in HTML, developer tools, and logs without risk.
components	['paypal-payments', 'paypal-guest-payments']	paypal-payments enables PayPal, PayLater, and PayPal Credit. paypal-guest-payments enables BCDC. For a complete integration, both.
pageType	'checkout'	Tells the SDK it operates on a checkout page (affects metrics and presentation).
locale	'es-MX', 'en-US', 'pt-BR', etc.	Language of the PayPal-hosted UI. BCP-47 format with hyphen. Must match the buyer's market.
clientMetadataId	The same CMID generated in §12 (optional).	Correlates the telemetry the SDK sends to PayPal Risk with the session, just like the server-side PayPal-Client-Metadata-Id headers.

 createInstance returns a Promise. It must be awaited (this occurs in the initialization phase, not in the click handler; there is no risk of consuming the transient activation).

15.2 sdkInstance.findEligibleMethods(...)

```
const paymentMethods = await sdkInstance.findEligibleMethods({ currencyCode:
  "<MONEDA_ISO_4217>" });

if (paymentMethods.isEligible("paypal")) renderPayPalButton();
if (paymentMethods.isEligible("paylater")) renderPayLaterButton();
if (paymentMethods.isEligible("credit")) {
  const creditDetails = paymentMethods.getDetails("credit");
  renderPayPalCreditButton(creditDetails);
}
```

```
if (paymentMethods.isEligible("guest")) renderGuestCardButton();
```

 Do not render a button whose method is not eligible. Doing so produces opaque errors in `session.start(...)` and damages the buyer experience.

 The keys accepted by `isEligible(...)` and `getDetails(...)` are: "paypal", "paylater", "credit" (not "paypal-credit"), and "guest". The marketing label "PayPal Credit" is kept only in the UI.

15.3 PayLater-specific details

```
const paylaterDetails = paymentMethods.getDetails("paylater");  
// paylaterDetails: { productCode: "<...>", countryCode: "<...>" }
```

16. HTML Structure and Session Activation Pattern

16.1 Merchant buttons

```
<!-- PayPal, PayLater, PayPal Credit: botones del comercio -->
<button id="btn-paypal" type="button">Pagar con PayPal</button>
<button id="btn-paylater" type="button">Pagar con PayLater</button>
<button id="btn-paypal-credit" type="button">Pagar con PayPal Credit</button>

<!-- BCDC: custom element hospedado por PayPal -->
<paypal-basic-card-container>
  <paypal-basic-card-button id="paypal-basic-card-button"></paypal-basic-card-button>
</paypal-basic-card-container>

<div id="payment-result" role="status" aria-live="polite"></div>
```

 For `<paypal-basic-card-container>` and `<paypal-basic-card-button>` to register correctly, the 'paypal-guest-payments' component must be included in the components array of `createInstance(...)`.

16.2 Session activation pattern — transient activation preservation

 This is the most critical rule of the integration. Any await before invoking `session.start(...)` consumes the state and the browser blocks the pop-up.

16.2.1 CORRECT pattern

```
document.getElementById("btn-paypal").addEventListener("click", () => {
  // Build the promise WITHOUT await – preserves transient activation
  const orderPromise = createOrder({ paymentMethod: "paypal" });
  session.start({ presentationMode: "auto" }, orderPromise).catch(onError);
});
```

16.2.2 INCORRECT pattern (do not implement)

```
// INCORRECT – the await consumes transient activation
document.getElementById("btn-paypal").addEventListener("click", async () => {
  const orderId = await createOrder({ paymentMethod: "paypal" }); // NUNCA HACER ESTO
  session.start({ presentationMode: "auto" }, Promise.resolve({ id: orderId }))
    .catch(onError);
});
```

16.3 BCDC pattern (click handler on custom element)


```
const cardBtn = document.getElementById("paypal-basic-card-button");
cardBtn.addEventListener("click", () => {
  const orderPromise = createOrder({ paymentMethod: "guest" });
  guestSession.start({ presentationMode: "auto" }, orderPromise).catch(onError);
});
```

💡 SDK v6 supports programmatic opening variants with `targetElement`. Those variants are not the canonical flow of this SDD; any deviation must be validated with the Integration Engineer.

16.4 Shape of the Create Order promise

```
async function createOrder({ paymentMethod }) {
  const response = await fetch("/api/orders", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      _cmid: cmid,
      paymentMethod: paymentMethod, // 'paypal' | 'paylater' | 'credit' | 'guest'
      cartId: getCurrentCartId()
    })
  });
  if (!response.ok) throw new Error(`Create Order failed: ${response.status}`);
  const order = await response.json();
  return { orderId: order.id }; // El SDK v6 exige la forma { orderId: <ORDER_ID> }
}
```

🔒 The promise must resolve to `{ orderId: <ORDER_ID> }`. Do not use `{ id: ... }`; SDK v6 specifically expects the `orderId` property. Returning the wrong shape causes `data.orderId` to arrive undefined in `onApprove`.

16.5 Session callbacks

Callback	PayPal / PayLater / Credit	BCDC	Function
onApprove(data)	Yes	Yes	The buyer approved the payment. <code>data.orderId</code> contains the <code>order_id</code> . The merchant fires the capture.
onCancel(data)	Yes	Yes	The buyer closed the UI without completing the payment. Show a sober message and allow retry.
onError(err)	Yes	Yes	Technical error during the session. Log and show an actionable message.

Callback	PayPal / PayLater / Credit	BCDC	Function
onWarn(warn)	—	Yes	Non-fatal warning specific to BCDC. The SDK already shows feedback to the user; the merchant typically only logs.

 In BCDC, do not omit onWarn. The callback exists specifically for warnings from the hosted UI; ignoring it leaves the merchant without telemetry on capture friction.

17–20. Payment Sessions

PayPal, PayLater, PayPal Credit, and BCDC sessions share the same pattern: initialization before the click, activation in the click handler with the Create Order promise, and capture in onApprove.

17. PayPal Checkout Session

```
const paypalSession = sdkInstance.createPayPalOneTimePaymentSession({
  onApprove: async (data) => {
    const result = await captureOrder(data.orderId);
    showSuccess(result);
  },
  onCancel: (data) => showInfo("Pago cancelado por el comprador."),
  onError: (err) => { console.error(err); showError("No fue posible completar el pago. Intente nuevamente."); }
});

document.getElementById("btn-paypal").addEventListener("click", () => {
  const orderPromise = createOrder({ paymentMethod: "paypal" });
  paypalSession.start({ presentationMode: "auto" }, orderPromise).catch(showError);
});
```

18. PayLater Session

```
if (!paymentMethods.isEligible("paylater")) {
  document.getElementById("btn-paylater").style.display = "none";
} else {
  const paylaterSession = sdkInstance.createPayLaterOneTimePaymentSession({
    onApprove: async (data) => {
      const result = await captureOrder(data.orderId);
      showSuccess(result);
    },
    onCancel: (data) => showInfo("Financiamiento cancelado."),
    onError: (err) => { console.error(err); showError("Error al iniciar PayLater."); }
  });
  document.getElementById("btn-paylater").addEventListener("click", () => {
    const orderPromise = createOrder({ paymentMethod: "paylater" });
    paylaterSession.start({ presentationMode: "auto" }, orderPromise).catch(showError);
  });
}
```

19. PayPal Credit Session

```
if (!paymentMethods.isEligible("credit")) {
  document.getElementById("btn-paypal-credit").style.display = "none";
} else {
```

```
const creditSession = sdkInstance.createPayPalCreditOneTimePaymentSession({
  onApprove: async (data) => {
    const result = await captureOrder(data.orderId);
    showSuccess(result);
  },
  onCancel: (data) => { showInfo("Operación cancelada."), },
  onError: (err) => { console.error(err); showError("Error al iniciar PayPal Credit."); },
});
document.getElementById("btn-paypal-credit").addEventListener("click", () => {
  const orderPromise = createOrder({ paymentMethod: "credit" });
  creditSession.start({ presentationMode: "auto" }, orderPromise).catch(showError);
});
```

20. BCDC Session — Guest Card Checkout

20.1 Key differences vs. the other sessions

Aspect	PayPal / PayLater / Credit	BCDC
Component in <code>createInstance</code>	'paypal-payments'	'paypal-guest-payments'
Constructor	<code>createPayPal*OneTimePaymentSession</code>	<code>createPayPalGuestOneTimePaymentSession</code>
Callbacks	<code>onApprove</code> , <code>onCancel</code> , <code>onError</code>	<code>onApprove</code> , <code>onCancel</code> , <code>onWarn</code> , <code>onError</code>
Element that fires the session	Merchant <code><button></code> HTML	<code><paypal-basic-card-button></code> (custom element registered by the SDK)
Card capture	Not applicable	PayPal-hosted UI

```
if (!paymentMethods.isEligible("guest")) {
  document.querySelector("paypal-basic-card-container").style.display = "none";
} else {
  const guestSession = sdkInstance.createPayPalGuestOneTimePaymentSession({
    onApprove: async (data) => {
      const result = await captureOrder(data.orderId);
      showSuccess(result);
    },
    onCancel: (data) => { showInfo("Pago con tarjeta cancelado."), },
    onWarn: (warn) => { console.warn("BCDC warning:", warn), },
    onError: (err) => { console.error(err); showError("No fue posible procesar la tarjeta."); },
  });
  document.getElementById("paypal-basic-card-button").addEventListener("click", () => {
    const orderPromise = createOrder({ paymentMethod: "guest" });
    guestSession.start({ presentationMode: "auto" }, orderPromise).catch(showError);
  });
}
```

💡 In the BCDC flow of SDK v6 covered by this SDD, the merchant does not receive or evaluate `liabilityShift` or `enrollment_status`. PayPal's eventual 3DS execution is transparent.

21. Order Creation and Capture

21.1 Required HTTP headers

Header	Create	Capture	Description
Authorization: Bearer <ACCESS_TOKEN>	Yes	Yes	Bearer token obtained in §11.
Content-Type: application/json	Yes	Yes	The body is JSON.
PayPal-Request-Id	<CREATE_REQUEST_ID>	<CAPTURE_REQUEST_ID>	Per-operation idempotency. Different key for Create and Capture. The same key is reused only for retries of the same request.
PayPal-Client-Metadata-Id: <CMID>	Yes	Yes	Links the transaction with the STC context. Same CMID in Create and Capture (session identifier, not operation identifier).

 The <CREATE_REQUEST_ID> and <CAPTURE_REQUEST_ID> keys are independent. Reusing the same key for Create and Capture violates PayPal's idempotency guidance and may produce inconsistent behavior.

21.2 Create Order — base payload

```
{
  "intent": "CAPTURE",
  "application_context": {
    "brand_name": "&lt;NOMBRE_VISIBLE_DEL_COMERCIO&gt;",
    "locale": "&lt;LOCALE_BCP47&gt;",
    "shipping_preference": "SET_PROVIDED_ADDRESS",
    "user_action": "PAY_NOW",
    "return_url": "&lt;URL_DE_RETORNO_REAL_DEL_COMERCIO&gt;",
```

```

    "cancel_url":          "<URL_DE_CANCELACION_REAL_DEL_COMERCIO>";
  },
  "payer": {
    "email_address": "<EMAIL_DEL_COMPRAADOR>";
    "name": { "given_name": "<NOMBRE_DEL_COMPRAADOR>", "surname":
    "<APELLIDO_DEL_COMPRAADOR>"; },
    "phone": { "phone_type": "MOBILE", "phone_number": { "national_number":
    "<TELEFONO_SOLO_DIGITOS>"; } }
  },
  "purchase_units": [{
    "invoice_id": "<INVOICE_ID_UNICO_DEL_COMERCIO>";
    "custom_id": "<ORDER_ID_INTERNO_DEL_COMERCIO>";
    "description": "<DESCRIPCION_BREVE_DEL_PEDIDO>";
    "amount": {
      "currency_code": "<MONEDA_ISO_4217>";
      "value": "<TOTAL_DEL_CARRITO>";
      "breakdown": {
        "item_total": { "currency_code": "<MONEDA>", "value":
        "<SUMA_UNIT_AMOUNT_X_QTY>"; },
        "tax_total": { "currency_code": "<MONEDA>", "value":
        "<SUMA_TAX_X_QTY>"; },
        "shipping": { "currency_code": "<MONEDA>", "value":
        "<COSTO_DE_ENVIO>"; },
        "discount": { "currency_code": "<MONEDA>", "value":
        "<DESCUENTO_APLICADO>"; }
      }
    },
    "items": [{
      "name": "<NOMBRE_DEL_PRODUCTO>";
      "description": "<DESCRIPCION_DEL_PRODUCTO>";
      "sku": "<SKU_DEL_CATALOGO>";
      "quantity": "<CANTIDAD>";
      "unit_amount": { "currency_code": "<MONEDA>", "value":
      "<PRECIO_UNITARIO_SIN_IMPUESTOS>"; },
      "tax": { "currency_code": "<MONEDA>", "value":
      "<IMPUESTO_POR_UNIDAD>"; },
      "category": "PHYSICAL_GOODS"
    }],
    "shipping": {
      "name": { "full_name": "<NOMBRE_COMPLETO_DEL_DESTINATARIO>"; },
      "address": {
        "address_line_1": "<CALLE_Y_NUMERO>";, "address_line_2":
        "<COLONIA_O_REFERENCIA>";,
        "admin_area_2": "<CIUDAD_O_MUNICIPIO>";, "admin_area_1":
        "<ESTADO_CODIGO>";,
        "postal_code": "<CODIGO_POSTAL>";, "country_code":
        "<PAIS_ISO_ALPHA2>";
      }
    }
  }
}

```

 The Create Order payload is identical for the four payment methods (PayPal, PayLater, PayPal Credit, BCDC). The merchant does not send payment_source. PayPal automatically associates the order with the payment method corresponding to the session constructor that was invoked.

21.3 Breakdown validation rules

```
amount.value === item_total + tax_total + shipping - discount
item_total    === Σ (item.unit_amount × item.quantity) por línea
tax_total     === Σ (item.tax × item.quantity) por línea
```

21.4 Capture Order

```
POST {PAYPAL_API_BASE}/v2/checkout/orders/&lt;ORDER_ID&gt;/capture
Authorization: Bearer &lt;ACCESS_TOKEN&gt;
Content-Type: application/json
PayPal-Request-Id: &lt;CAPTURE_REQUEST_ID&gt;
PayPal-Client-Metadata-Id: &lt;CMID&gt;
{}
```

21.5 Frontend → backend contract

 The frontend does not build the Create Order payload. It sends only controlled references: `_cmid`, `paymentMethod`, and `cartId`. The backend reconstructs the payload from the authenticated cart's trusted state. Accepting amounts sent from the client exposes the merchant to price manipulation.

21.6 Backend — POST /api/orders

```
app.post("/api/orders", async (req, res) => {
  const { _cmid, paymentMethod, cartId } = req.body;

  // 1) Input validation
  if (!isValidCMID(_cmid)) return res.status(400).json({ error:
"invalid cmid" });
  if (!isAllowedPaymentMethod(paymentMethod)) return res.status(400).json({ error:
"invalid paymentMethod" });
  const cart = await loadCartForUser(cartId, req.user); // valida ownership
  if (!cart) return res.status(404).json({ error: "cart
not found" });

  // 2) STC server-side, secuenciada, no bloqueante
  try {
    await callSTC({ cmid: _cmid, additionalData:
buildAdditionalDataFromTrustedState(req.user) });
  } catch (err) {
    logger.warn({ err, cmid: _cmid }, "STC failed; continuing checkout");
  }

  // 3) Build payload from trusted state
  const orderPayload = buildOrderPayloadFromTrustedState({ cart, user: req.user });
  const accessToken = await getAccessToken();
  const createRequestId = await getOrCreateCreateRequestId(cartId);
```



```
// 4) Create Order with per-operation idempotency
const response = await fetch(`${process.env.PAYPAL_API_BASE}/v2/checkout/orders`, {
  method: "POST",
  headers: {
    "Authorization": `Bearer ${accessToken}`,
    "Content-Type": "application/json",
    "PayPal-Request-Id": createRequestId,
    "PayPal-Client-Metadata-Id": _cmid
  },
  body: JSON.stringify(orderPayload)
});

const body = await response.json();
if (response.ok) await persistOrderContext({ orderId: body.id, cartId, cmid: _cmid,
createRequestId });
res.status(response.status).json(body);
});
```

Part IV — Integration and Operation

22. End-to-End Orchestration

22.1 Moment 1 — Checkout initialization (once per session)

```

1. cmid = generateCMID() [§12]
2. { clientId, sdkUrl } = GET /api/config [§10.4]
3. await loadSDK(sdkUrl) [§14.3]
4. sdkInstance = await paypal.createInstance({ clientId, components,
                                           pageType, locale }) [§15.1]
5. paymentMethods = await sdkInstance.findEligibleMethods({ [§15.2]
                                                           currencyCode })
6. Si isEligible('paylater'): paylaterDetails = getDetails('paylater') [§15.3]
7. Construir sesiones con sus callbacks: [§17-20]
   paypalSession / paylaterSession / creditSession / guestSession
8. Renderizar solo botones cuyo método es elegible. [§16.1]
9. Adjuntar addEventListener('click', ...) con patrón sin await. [§16.2]

```

22.2 Moment 2 — Buyer click (no await before start)

```

btn.addEventListener("click", () => {
  const orderPromise = createOrder({ paymentMethod }); // sin await
  session.start({ presentationMode: "auto" }, orderPromise).catch(onError);
});

// createOrder(...) internally calls POST /api/orders, which executes in order:
// 1) Validate _cmid, paymentMethod, and cartId ownership.
// 2) await callSTC({ cmid, additionalData }) ← try/catch, no bloqueante
// 3) Construir orderPayload desde estado confiable
// 4) createRequestId = getOrCreateCreateRequestId(cartId)
// 5) POST /v2/checkout/orders + headers
// 6) Persist { orderId, cartId, _cmid, createRequestId }
// 7) Respond to the frontend with { id: orderId }

```

22.3 Moment 3 — onApprove and capture

```

onApprove(data) {
  const result = await captureOrder(data.orderId);
  showSuccess(result);
}

// POST /api/orders/<ORDER_ID>/capture + body._cmid = cmid
// → backend:
//   captureRequestId = getOrCreateCaptureRequestId(orderId)
//   headers: PayPal-Request-Id: <CAPTURE_REQUEST_ID> (different from Create's)
//           PayPal-Client-Metadata-Id: <CMID> (same as the session's)

```

22.4 Mnemonic rule

Component	Frequency
CMID	Once per checkout session. Reused in STC, Create, and Capture.
createInstance + findEligibleMethods	Once per checkout session (after loading the SDK).
Server-side STC	Before each Create Order, inside the POST /api/orders handler.
createRequestId	One UUID per Create Order intent (per cartId). Reused for retries of the same Create.
captureRequestId	One UUID per Capture Order intent (per orderId). Reused for retries of the same Capture. Distinct from createRequestId.
PayPal-Client-Metadata-Id	Same CMID in Create and Capture.
session.start(...)	Once per charge attempt, inside the click handler, with no prior await.

23. Integration Points (Endpoint Map)

Operation	Merchant backend (suggested)	PayPal API
Public configuration for the frontend	GET /api/config	— (does not reach PayPal)
OAuth token (backend internal use)	(internal)	POST /v1/oauth2/token
Set Transaction Context (internal in POST /api/orders)	(internal)	PUT /v1/risk/transaction-contexts/{merchant_id}/{cmid}
Create order (includes server-side STC)	POST /api/orders	POST /v2/checkout/orders
Query order	GET /api/orders/:id	GET /v2/checkout/orders/{id}
Capture order	POST /api/orders/:id/capture	POST /v2/checkout/orders/{id}/capture

💡 No public /api/stc endpoint is exposed to the frontend. STC runs server-side inside POST /api/orders to guarantee the strict STC → Create Order order and so that the frontend has no knowledge of the risk flow.

24. Security Considerations

24.1 Credential custody (NFR-02)

Asset	Storage	Transmission
PAYPAL_CLIENT_SECRET	Backend environment variable, managed by a secrets manager. Never in source code.	Backend only → /v1/oauth2/token encoded in Authorization: Basic.
PAYPAL_MERCHANT_ID	Backend environment variable.	Backend only → STC URL.
access_token	Backend process memory. Cache with TTL lower than expires_in. Never persisted.	Backend only → PayPal REST API in Authorization: Bearer.

24.2 PCI isolation (NFR-01)

- In PayPal, PayLater, and Credit, the flow does not involve buyer cards from the merchant's perspective.
- In BCDC, the buyer's card account data is captured entirely in the PayPal-hosted UI. It never enters the merchant's DOM or backend.
- The merchant can certify under SAQ A.

24.3 Minimum recommended CSP (NFR-04)

```
script-src 'self' https://www.paypal.com https://www.paypalobjects.com
https://c.paypal.com;
frame-src https://www.paypal.com;
connect-src 'self' https://api-m.paypal.com https://api-m.sandbox.paypal.com;
img-src 'self' https://www.paypalobjects.com data:;
style-src 'self' https://www.paypalobjects.com 'unsafe-inline';
```

💡 In Sandbox environments, additionally add `https://www.sandbox.paypal.com` to `script-src` and `frame-src`.

24.4 Logging and sensitive data handling (NFR-08)

Allowed in logs:

- `order_id`, `capture_id`, `invoice_id`, `custom_id`, `CMID`, `createRequestId`, `captureRequestId`.
- capture status, PayPal error code, and validation messages.
- Name of the chosen payment method (paypal, paylater, credit, guest).

Forbidden in logs:

- access_token, client_secret, merchant_id, full Authorization header.
- PayPal response bodies with unmasked secrets.

25. Operational Considerations

25.1 Metrics and observability

Metric	Granularity	Suggested alarm
Success rate of POST /v1/oauth2/token	Per minute	< 99% in a 5-minute window.
p95 latency of Create Order and Capture Order	Per minute	Exceeds internal SLO.
Response rate $\neq$ 200 in STC	Per minute	> 5% in a 5-minute window.
Capture success rate per method (PayPal, PayLater, Credit, BCDC)	Per hour	Anomaly vs historical baseline.
onCancel rate per method	Per hour	Unusual spike suggests UI friction.
onError rate per method	Per hour	Indicates technical issue (SDK, network, configuration).
UNPROCESSABLE_ENTITY rate in Create Order	Per minute	> 0.5% indicates bug in breakdown construction.

25.2 Idempotency (NFR-06)

PayPal applies idempotency per operation: each REST API call has its own PayPal-Request-Id key, and the same key is reused only for retries of the same request.

Operation	Recommended key	Reuse
Create Order	createRequestId (server-side UUID, per cartId)	Retries of the same Create (timeouts, network errors, transient 5xx).
Capture Order	captureRequestId (server-side UUID, per orderId)	Retries of the same Capture.
New order	New createRequestId	Even though the CMID may be preserved in the same session.
New capture on a new order	New captureRequestId	—

```
async function getOrCreateCreateRequestId(cartId) {
  const existing = await store.get(`create-request-id:${cartId}`);
  if (existing) return existing;
  const id = crypto.randomUUID();
  await store.set(`create-request-id:${cartId}`, id, { ttl: CHECKOUT_TTL });
}
```

```

    return id;
  }

  async function getOrCreateCaptureRequestId(orderId) {
    const existing = await store.get(`capture-request-id:${orderId}`);
    if (existing) return existing;
    const id = crypto.randomUUID();
    await store.set(`capture-request-id:${orderId}`, id, { ttl: ORDER_TTL });
    return id;
  }

```

 Do not use the same UUID for Create and Capture. Per-operation separation aligns with PayPal's official idempotency guidance and prevents undefined behavior.

25.3 Fault tolerance

Service	Strategy
POST /v1/oauth2/token	Retry once after 401/5xx; if it persists, abort and alert.
POST /v2/checkout/orders	Retry with the same createRequestId on 5xx or transient network errors.
POST /v2/checkout/orders/{id}/capture	Same as Create Order (same captureRequestId).
PUT /v1/risk/transaction-contexts/... (STC)	Do not retry. Log the failure and continue the checkout.
Loading SDK v6	If the <script> fails to load, disable payment buttons and show a degradation message to the user.

25.4 Sandbox → Live promotion

Point	Change
PAYPAL_API_BASE	https://api-m.sandbox.paypal.com → https://api-m.paypal.com
PAYPAL_SDK_URL	https://www.sandbox.paypal.com/web-sdk/v6/core → https://www.paypal.com/web-sdk/v6/core
PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET	Live app credentials, not Sandbox.
PAYPAL_MERCHANT_ID	Live account Merchant ID.
application_context.return_url / cancel_url	Real URLs of the merchant's production domain.

Point	Change
CSP	Add/remove Sandbox domains as appropriate.

Part V — Validation and Governance

26. Testing Strategy and Sandbox Environment

26.1 Minimum test cases (TC-01 to TC-16)

ID	Case	Expected result	Meets FR/NFR
TC-01	Payment with PayPal account (valid Sandbox account)	Successful capture, <code>purchase_units[0].payments.captures[0].status === "COMPLETED"</code> .	FR-01, FR-04, FR-05
TC-02	Payment with PayLater in eligible market	Successful capture.	FR-03, FR-05
TC-03	Payment with PayPal Credit in eligible market	Successful capture.	FR-03, FR-05
TC-04	Card payment via BCDC (valid Sandbox card)	Successful capture. Inspection confirms no merchant payload contains card account data.	FR-02, FR-04, FR-05, NFR-01
TC-05	Cancellation by the buyer (onCancel)	UI shows a sober message; capture is not invoked.	FR-11
TC-06	Technical error during the session (onError)	UI shows an actionable message, log with error detail.	FR-11
TC-07a	Technical retry of Create due to timeout/5xx (same cartId)	Same CMID, same createRequestId, same orderId returned (idempotent operation).	NFR-06
TC-07b	New buyer attempt after rejection or cancellation	Same CMID if still in the same session, new createRequestId, new order.	FR-09
TC-08	Idempotent retry of Capture (same captureRequestId)	PayPal returns the previous capture, does not generate a duplicate.	NFR-06
TC-09	STC returns 400	Checkout is not interrupted; order is created correctly.	FR-07, NFR-05
TC-10	Eligibility: market without PayLater	PayLater button is not rendered.	FR-12, NFR-12
TC-11	Eligibility: currency without PayPal Credit	PayPal Credit button is not rendered.	FR-12, NFR-12
TC-12	Transient activation validation	PayPal pop-up/modal opens without blocking in Chrome, Firefox, Safari, and Edge.	FR-06, NFR-14

ID	Case	Expected result	Meets FR/NFR
TC-13	GET /api/config does not expose secrets	Response contains clientId and sdkUrl; does not contain client_secret, merchant_id, or access_token.	NFR-02
TC-14	CSP validation	SDK load and opening of the hosted UI produce no CSP violations.	NFR-04
TC-15	Sandbox → Live switch	After changing environment variables and reloading the page, the SDK operates against the correct environment.	\$25.4, \$25.6
TC-16	CMID validation	Inspection confirms the same CMID in STC, Create Order's PayPal-Client-Metadata-Id, and Capture Order's.	FR-08

27. REST vs SDK Naming Conventions

Concept	REST API (snake_case)	JavaScript SDK v6 (camelCase)
Order identifier	id (in response) / order_id	orderId (in data.orderId of onApprove)
Currency code	currency_code	currencyCode
Country code	country_code	countryCode
SDK component	(not applicable in REST)	components: ['paypal-payments', 'paypal-guest-payments']
Page type	(not applicable in REST)	pageType: 'checkout'
PayPal client identifier	client_id (OAuth form-encoded)	clientId (in createInstance)
UI presentation mode	(not applicable in REST)	presentationMode: 'auto'
Risk correlation header	PayPal-Client-Metadata-Id (HTTP header)	(not applicable in SDK; injected server-side)
Idempotency header	PayPal-Request-Id (HTTP header, key per operation)	(not applicable in SDK; injected server-side)

 Mental rule: If the data leaves or enters a REST endpoint (JSON body or HTTP headers), it is snake_case or Pascal-Case with hyphens (in headers). If you pass it to a paypal.* object method or receive it in an SDK callback, it is camelCase.

28. Assumptions, Dependencies, and Constraints

28.1 Assumptions

ID	Assumption
A-01	The merchant has a backend under its control where it can custody PAYPAL_CLIENT_SECRET, PAYPAL_MERCHANT_ID, and issue the access_token.
A-02	The merchant has a user authentication system and a cart service that produces a mathematically consistent breakdown.
A-03	The target market supports the configured currency and the eligibility of the desired payment methods (PayPal, PayLater, Credit, BCDC).
A-04	The buyer uses a modern browser with support for pop-ups, transient activation, ES2017+, and CSP.
A-05	The PayPal account has the commercial enablements listed in §10.1 activated.
A-06	The merchant manages the Sandbox → Live environment switch via backend environment variables; runtime alternation is not required.

28.2 External dependencies

ID	Dependency	Type
D-01	PayPal REST API (api-m.paypal.com, api-m.sandbox.paypal.com).	Critical — blocking.
D-02	PayPal Web SDK v6 (https://www.paypal.com/web-sdk/v6/core and its Sandbox equivalent).	Critical — blocking.
D-03	OAuth2 endpoint (/v1/oauth2/token).	Critical — without it there is no access_token.
D-04	STC endpoint (/v1/risk/transaction-contexts).	Recommended — non-blocking.
D-05	Domains www.paypal.com, www.paypalobjects.com, c.paypal.com (and Sandbox equivalents) accessible from the buyer's browser and allowed in CSP.	Critical — the hosted UI lives on these domains.

28.3 Constraints

ID	Constraint
R-01	SDK v6 does not accept query-string parameters in its URL nor data-* configuration attributes. All configuration goes through <code>createInstance(...)</code> .
R-02	SDK v6 registers custom elements and cannot be re-initialized on the same page. Switching environments (Sandbox ↔ Live) requires reloading.
R-03	The <code>createOrder</code> callback cannot be awaited inside the click handler; the mandatory pattern is to build the promise and immediately pass it to <code>session.start(...)</code> .
R-04	The canonical BCDC flow is activated with <code>addEventListener('click', ...)</code> on <code><paypal-basic-card-button></code> , without <code>targetElement</code> . The variant with <code>targetElement</code> (programmatic opening / auto-start) requires validation with the Integration Engineer.
R-05	The CMID is unique per checkout session and must not be reused across buyers or different sessions.
R-06	PayPal-Request-Id is managed with independent keys per operation: <code>createRequestId</code> for Create Order, <code>captureRequestId</code> for Capture Order. Each key is reused only for retries of the same request.
R-07	<code>client_secret</code> and <code>merchant_id</code> must not, under any circumstance, reach the browser.
R-08	The integration covered by this SDD uses exclusively intent: "CAPTURE". AUTHORIZE is out of scope.

29. Risks and Mitigations

ID	Risk	Prob.	Impact	Mitigation
RG-01	Exposure of PAYPAL_CLIENT_SECRET via accidental commit to the repository.	Medium	Critical	Pre-commit hooks that detect secret patterns; secrets manager; mandatory security review. Immediate rotation plan if exposure is detected.
RG-02	Exposure of access_token to the frontend via a bug in GET /api/config or in the /api/* response wrapper.	Low	Critical	Automated tests that validate responses do not contain access_token, client_secret, or merchant_id; mandatory code review.
RG-03	Pop-up blocked by accidental consumption of transient activation (use of await before session.start).	High	High	Code review with explicit checklist; custom lint rule; E2E test that confirms pop-up opens without requiring additional user interaction.
RG-04	BCDC with incorrect markup (merchant's own <button> HTML instead of <paypal-basic-card-button>, or 'paypal-guest-payments' component absent from createInstance).	Medium	High	E2E test verifying that <paypal-basic-card-button> is present, hydrated, and fires the session.
RG-05	Mixing Sandbox/Live credentials (cross-environment).	Low	Critical	Validation of the CLIENT_ID prefix and PAYPAL_API_BASE at backend startup; alarm if inconsistent.
RG-06	Duplicate Order or Capture due to retry without idempotency, or key reused across operations.	Medium	High	Separate keys per operation (createRequestId, captureRequestId), persisted server-side. Test validating that Create and Capture keys are distinct.
RG-07	Non-eligible method buttons rendered to the buyer.	Medium	Medium	Mandatory filtering by findEligibleMethods before rendering. E2E test per market.
RG-08	CSP failure that blocks loading of the SDK or hosted UI.	Medium	High	CSP tested in staging per environment; detection fallback and clear user message if scripts fail to load.

ID	Risk	Prob.	Impact	Mitigation
RG-09	Checkout blocked by STC error (NFR-05 violation).	Low	Critical	STC implemented as non-blocking with try/catch (§13.5). TC-09 test.
RG-10	SDK initialization against the wrong environment (mixed URL or client_id).	Low	Critical	GET /api/config is the single source of clientId and sdkUrl for the frontend; the backend validates consistency among PAYPAL_API_BASE, PAYPAL_SDK_URL, and credentials.
RG-11	SDK re-initialization on the same page when switching environment → custom-elements conflict.	Low	High	Force page reload on environment change; document the constraint for the operations team.
RG-12	Inconsistent breakdown → 422 UNPROCESSABLE_ENTITY.	Medium	Medium	Pure function in the backend that assembles breakdown and items with mathematical validation before sending.
RG-13	Logs leak backend secrets (access_token, client_secret) without masking.	Medium	Critical	Logging wrapper that applies default masking to sensitive keys; observability pipeline review.
RG-14	The merchant changes the clientId without reloading the page → SDK left with stale instance.	Low	Medium	clientId changes always trigger a reload (location.reload() or navigation).

30. Acceptance Criteria and Pre-production Checklist

30.1 Security

- ☐ PAYPAL_CLIENT_SECRET and PAYPAL_MERCHANT_ID reside only in backend environment variables; they are never included in versioned source code or sent to the browser.
- ☐ Configuration files with credentials (.env, equivalents) excluded from version control.
- ☐ The backend exposes only clientId and sdkUrl to the frontend via GET /api/config; never access_token, client_secret, or merchant_id.
- ☐ Logs do not record access_token, client_secret, full Authorization headers, or sensitive fields without masking.
- ☐ Content Security Policy allows <https://www.paypal.com>, <https://www.paypalobjects.com>, and <https://c.paypal.com>.
- ☐ TLS 1.2+ enabled in production.

30.2 Configuration

- ☐ PAYPAL_API_BASE points to <https://api-m.paypal.com> in Live.
- ☐ PAYPAL_SDK_URL points to <https://www.paypal.com/web-sdk/v6/core> in Live.
- ☐ PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET, and PAYPAL_MERCHANT_ID match the Live environment.
- ☐ application_context.return_url and cancel_url are real, accessible URLs of the merchant's domain.
- ☐ The SDK's <script> includes no data-* attributes or query-string parameters.
- ☐ The createInstance components array contains exactly the components required for the exposed methods.

30.3 Functional

- ☐ findEligibleMethods is invoked on every checkout initialization and only buttons whose methods are eligible are rendered.
- ☐ Each button's click handler contains no await before session.start(...).
- ☐ BCDC: HTML contains <paypal-basic-card-container> with <paypal-basic-card-button> inside; the click listener is on the <paypal-basic-card-button>.
- ☐ breakdown and items reflect the actual cart state and meet the rules of §21.3.
- ☐ PayPal-Request-Id: independent keys per operation (createRequestId and captureRequestId). The same key is never reused between Create and Capture.
- ☐ Error handling implemented in onError, onCancel, and (in BCDC) onWarn.
- ☐ payer, shipping, and purchase_units data come from the checkout state, not from placeholders.
- ☐ After capture, the back office records order_id, capture_id, status, CMID, createRequestId, captureRequestId, invoice_id, and custom_id.

30.4 Risk

- ☐ CMID generated once per checkout session, propagated between frontend and backend.
- ☐ STC is called before each Create Order (regardless of method).
- ☐ STC additional_data comes from the buyer's authenticated session.
- ☐ STC does not block the checkout: errors are logged and the transaction continues.
- ☐ PayPal-Client-Metadata-Id header present in Create Order and Capture Order, with the same CMID.

30.5 Testing

- ☐ Test case matrix TC-01 to TC-16 executed and green in Sandbox.
- ☐ DevTools inspection confirms that no browser call goes directly to api-m.paypal.com.
- ☐ DevTools inspection confirms that GET /api/config does not expose access_token, client_secret, or merchant_id.
- ☐ Manual tests on Chrome, Firefox, Safari, and Edge confirm the pop-up/modal opens without blocking on the four methods.

30.6 Operation

- ☐ \$25.1 metrics instrumented and connected to the observability system.
- ☐ Alarms configured for the suggested thresholds.
- ☐ Incident runbook documented: OAuth failure, Create Order failure, anomaly in capture rates per method, SDK outage.
- ☐ CLIENT_SECRET rotation plan documented and tested in Sandbox.
- ☐ Procedure documented for Sandbox → Live promotion and for coordinated change of environment variables.

Appendix A — Common Errors Diagnostic

Symptom	Most likely cause	How to diagnose
401 Unauthorized on /v1/oauth2/token	CLIENT_ID / CLIENT_SECRET miscopied or cross-environment (Sandbox vs Live).	Verify that PAYPAL_API_BASE, PAYPAL_SDK_URL, and credentials match the same environment.
The PayPal pop-up is blocked by the browser	The click handler contains await before session.start(...), consuming transient activation.	Inspect the handler: the Create Order promise must be built and immediately passed to session.start(...) with no prior await.
BCDC: <paypal-basic-card-button> does not render or is inert	The 'paypal-guest-payments' component is not included in createInstance(...) components, or the HTML was mounted before the SDK finished loading.	Confirm that 'paypal-guest-payments' is in components. Ensure createInstance(...) is invoked before showing the container.
BCDC: the click does not fire the session	The listener was attached to a wrong element (e.g., to <paypal-basic-card-container> instead of <paypal-basic-card-button>), or the element was replaced by the SDK after attaching the listener.	Attach the listener to <paypal-basic-card-button> after the SDK has finished hydrating the custom elements.
findEligibleMethods returns an empty set	client_id + currencyCode + buyer country combination without sufficient enablements.	Verify the commercial enablement in the Developer Dashboard; verify ISO 4217 currencyCode; consult the Integration Engineer.
PayLater button rendered but the session fails to activate	It was rendered without validating paymentMethods.isEligible('paylater').	Filter by isEligible(...) before showing each button.
422 UNPROCESSABLE_ENTITY on Create Order	breakdown that doesn't add up with amount.value and/or items sum.	Apply the rules of §21.3 manually; PayPal returns the specific inconsistent field in the response.
Capture executes twice on a browser retry	The backend generates a new captureRequestId on each call instead of looking it up in its store by orderId.	Implement getOrCreateCaptureRequestId(or derId) with persistence to return the same UUID on retries of the same Capture.
Contradictory messages when retrying Create after a timeout	The backend generates a new createRequestId on the retry →	Persist createRequestId by cartId and reuse it on retries of the same Create.

Symptom	Most likely cause	How to diagnose
	PayPal creates an additional order.	
Environment switch (Sandbox ↔ Live) leaves the page broken	Attempted to re-initialize the SDK without reloading the page.	Force a reload (location.reload()) after changing environment variables.
GET /api/config returns access_token or client_secret	Bug in the endpoint implementation: the entire configuration object is serialized.	Implement a field whitelist ({ clientId, sdkUrl }) instead of res.json(config) directly.
STC returns 400	Incorrect field type in additional_data.	Validate formats per §13.3 (especially sender_create_date and sender_phone).
STC returns 5xx but the checkout is interrupted	The POST /api/orders handler is propagating the STC error instead of absorbing it inside its try/catch.	The await callSTC(...) block must be wrapped in try/catch; any exception is logged as a warn and Create Order proceeds.
paypal.createInstance is not a function	The SDK's <script> did not finish loading before the call.	Wait for the <script>'s onload or use the loadSDK(...) helper with Promise.
onApprove receives data.orderId undefined	Incorrect shape of the promise returned by createOrder.	The callback must return { orderId: <ORDER_ID> } (not { id: ... } or the bare order_id).
The SDK does not show the UI in the expected language	Malformed or unsupported locale in the buyer's market.	Use BCP-47 format with hyphen ('es-MX', not 'es_MX').

Appendix B — Limitations and Future Work

B.1 Known limitations

- Runtime environment switching is not supported by SDK v6 (registers custom elements). The Sandbox/Live selection must be made at backend startup.
- AUTHORIZE + deferred capture is out of scope. This integration uses exclusively intent: "CAPTURE".
- Advanced card capabilities (capture in merchant-owned iframes, explicit control of step-up authentication and its outcome, installments, card reuse across sessions) are not part of this integration.
- The targetElement variant for programmatic BCDC activation (auto-start) is not the canonical flow of this SDD and requires validation with the Integration Engineer.

B.2 Suggested future work

Initiative	Description
Back-office operations	Complementary SDD for webhooks (PAYMENT.CAPTURE.COMPLETED, PAYMENT.CAPTURE.DENIED, CUSTOMER.DISPUTE.CREATED), partial/full refunds, settlement, and financial reconciliation.
Additional payment methods	Extension of the solution to include local wallets or other methods available in SDK v6, adding the corresponding components to createInstance.
Deferred capture (AUTHORIZE)	For business models (preorders, dropshipping) where capture occurs after inventory or shipping confirmation.
Industry pack	Adoption of the extended STC additional_data set matching the merchant's vertical (Travel, Gaming, Financial Services, etc.).

Solution Design Document for the integration of PayPal Checkout (PayPal, PayLater, PayPal Credit) and Branded Card-Direct Checkout (BCDC) on top of the PayPal JavaScript SDK v6 with server-side architecture.